

Thread Concurrency Notes for Placements

1. Big Picture

Concurrency means a system can make progress on more than one task during the same period of time.

Parallelism means tasks are literally running at the same instant, usually on different CPU cores.

They are related, but not the same.

Example:

- A single-core CPU can be concurrent by switching quickly between tasks.
- A multi-core CPU can be parallel by running multiple tasks at the same time.

In operating systems, concurrency is mainly achieved using:

- Processes
- Threads
- CPU scheduling
- Synchronization mechanisms
- Inter-process communication
- Shared memory
- Locks, semaphores, and condition variables

The main problem with concurrency is that multiple execution flows may access shared data. If they do so without coordination, the program may behave unpredictably.

2. Process

A process is a program in execution.

A program is passive: it is just code stored on disk.

A process is active: it has memory, CPU state, resources, and execution progress.

Process Contains

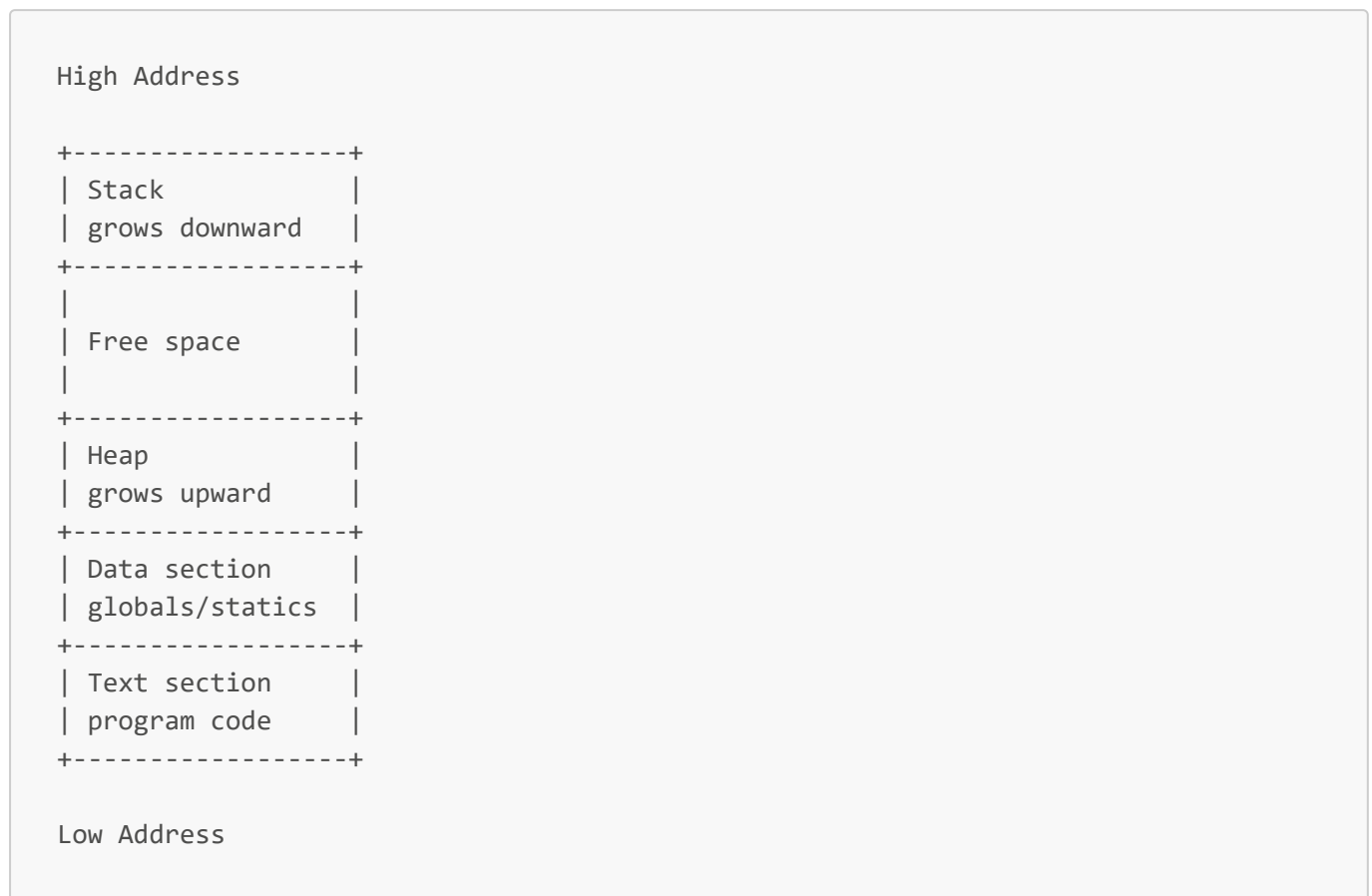
A process usually contains:

- Program code, also called the text section
- Data section for global and static variables
- Heap for dynamic memory allocation
- Stack for function calls and local variables
- Program counter
- CPU registers
- Open files
- Process ID
- Scheduling information

- Security information

Process Memory Layout

Typical process memory layout:



Process Control Block

The operating system stores information about each process in a data structure called the Process Control Block, or PCB.

The PCB contains:

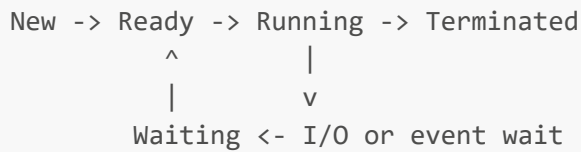
- Process ID
- Process state
- Program counter
- CPU registers
- CPU scheduling information
- Memory management information
- Accounting information
- I/O status information

When the CPU switches from one process to another, the OS saves the current process state in its PCB and loads the next process state from its PCB.

This is called a context switch.

Process States

Common process states:



Meaning:

- New: process is being created.
- Ready: process is waiting for CPU.
- Running: process is currently executing.
- Waiting: process is waiting for I/O or some event.
- Terminated: process has finished execution.

Process Advantages

- Strong isolation between processes
- One process crash usually does not directly corrupt another process
- Better security boundaries
- Useful for independent applications

Process Disadvantages

- Process creation is expensive
- Context switching between processes is expensive
- Communication between processes is slower
- Each process has its own address space, so sharing data needs special mechanisms

3. Thread

A thread is the smallest unit of CPU execution within a process.

A process can have one thread or many threads.

Threads inside the same process share the process resources, but each thread has its own execution state.

Threads Share

Threads of the same process share:

- Code section
- Data section
- Heap
- Open files
- Process address space
- Process resources

Each Thread Has Its Own

Each thread has:

- Thread ID
- Program counter
- Register set
- Stack
- Scheduling state

Thread Diagram



Why Use Threads?

Threads are useful because:

- They improve responsiveness.
- They allow resource sharing.
- They are cheaper than processes.
- They can use multiple CPU cores.
- They help structure programs that perform multiple activities.

Example:

A web browser may use different threads for:

- User interface
- Network loading
- Rendering
- JavaScript execution
- File downloads

Thread Advantages

- Faster creation than processes
- Faster context switching than processes
- Easy communication because memory is shared
- Efficient use of multi-core processors

Thread Disadvantages

- Bugs are harder to debug
- Shared memory can cause race conditions
- One bad thread can corrupt the whole process
- Synchronization is required
- Deadlocks are possible

4. Process vs Thread

Feature	Process	Thread
Basic meaning	Program in execution	Execution unit inside a process
Address space	Separate address space	Shares process address space
Memory sharing	Difficult, needs IPC	Easy, shared memory
Creation cost	High	Low
Context switch cost	Higher	Lower
Communication	Slower	Faster
Isolation	Strong	Weak
Crash impact	Usually affects only that process	Can affect entire process
Security	Better isolation	Less isolation
Resource ownership	Owns resources	Uses process resources
Example	Chrome process, terminal process	UI thread, worker thread

Placement-Friendly Answer

A process is an independent program in execution with its own address space and resources. A thread is a lightweight execution unit inside a process. Threads of the same process share code, data, heap, and open files, but each thread has its own stack, registers, and program counter. Processes provide better isolation but are expensive to create and switch. Threads are faster and easier for communication, but they require synchronization because they share memory.

5. Single-Threaded vs Multi-Threaded Process

Single-Threaded Process

A single-threaded process has only one execution flow.

If that thread blocks, the whole process blocks.

Example:

```
Process
|
+-- Thread 1
```

Multi-Threaded Process

A multi-threaded process has multiple execution flows.

If one thread waits for I/O, another thread may continue executing.

Example:

```
Process
|
+-- Thread 1
+-- Thread 2
+-- Thread 3
```

Benefits of Multithreading

- Responsiveness
- Resource sharing
- Economy
- Scalability

Example

In a word processor:

- One thread handles typing.
- One thread checks spelling.
- One thread auto-saves the document.
- One thread handles printing.

6. User Threads and Kernel Threads

Threads can be implemented at the user level, kernel level, or both.

User-Level Threads

User-level threads are managed by a user-level thread library, not directly by the operating system kernel.

The kernel is unaware of individual user threads. It only sees the process.

User-Level Thread Advantages

- Very fast to create and manage
- Thread switching does not require kernel mode
- Can use custom scheduling
- Works even if OS does not support threads

User-Level Thread Disadvantages

- If one thread performs a blocking system call, the entire process may block.
- Kernel cannot schedule individual user threads on multiple cores.
- Kernel sees only one process, not multiple threads.

Kernel-Level Threads

Kernel-level threads are managed directly by the operating system.

The kernel knows about each thread and schedules them individually.

Kernel-Level Thread Advantages

- If one thread blocks, other threads in the same process can continue.
- Threads can run in parallel on multiple CPU cores.
- Kernel can schedule each thread independently.

Kernel-Level Thread Disadvantages

- Creation and management are slower than user-level threads.
- Thread switching requires kernel involvement.
- More overhead.

User Threads vs Kernel Threads

Feature	User-Level Threads	Kernel-Level Threads
Managed by	User-level library	Operating system kernel
Kernel awareness	Kernel does not know individual threads	Kernel knows each thread
Creation speed	Faster	Slower
Context switch	Faster	Slower
Blocking system call	May block entire process	Blocks only calling thread
Parallel execution	Usually not true parallelism	Can run on multiple cores
Scheduling	Done by library	Done by kernel

7. Multithreading Models

Multithreading models describe how user threads are mapped to kernel threads.

7.1 Many-to-One Model

Many user threads are mapped to one kernel thread.

```
User threads:  T1  T2  T3  T4
                \  |  |  /
Kernel thread:   K1
```

Advantages:

- Simple
- Fast thread management

Disadvantages:

- One blocking system call blocks all threads.
- No true parallelism on multiple cores.

7.2 One-to-One Model

Each user thread maps to one kernel thread.

```
User threads:  T1  T2  T3
                |  |  |
Kernel threads: K1  K2  K3
```

Advantages:

- Better concurrency
- Threads can run on multiple cores
- One blocking thread does not block all threads

Disadvantages:

- More overhead
- OS may limit number of threads

7.3 Many-to-Many Model

Many user threads are mapped to many kernel threads.

```
User threads:  T1 T2 T3 T4 T5
                \ | X | /
Kernel threads:  K1 K2 K3
```

Advantages:

- Flexible
- Better concurrency
- Avoids creating too many kernel threads

Disadvantages:

- Complex implementation
-

8. Context Switching

Context switching means stopping one running execution unit and starting another.

The OS saves the state of the current process or thread and loads the state of another.

What Is Saved During a Context Switch?

Usually:

- Program counter
- CPU registers
- Stack pointer
- Process or thread state
- Memory management information
- Scheduling information

Process Context Switch

Switching between processes is expensive because:

- Address space may change.
- Memory mappings may change.
- CPU cache may be disturbed.
- Translation Lookaside Buffer, or TLB, may need updates.

Thread Context Switch

Switching between threads of the same process is usually cheaper because:

- Address space is shared.
- Less memory management state changes.
- Shared data remains accessible.

However, thread switching still has overhead.

9. Race Condition

A race condition occurs when the correctness of a program depends on the timing or ordering of execution of multiple threads or processes.

In simple words:

If two or more threads access shared data at the same time, and at least one thread modifies it, the final result may be wrong unless synchronization is used.

Example

Suppose two threads increment a shared variable `count`.

```
count++;
```

This line looks like one operation, but internally it may be:

1. Read `count` from memory into register
2. Add 1
3. Write result back to memory

Assume `count = 5`.

Thread A:

```
Read count = 5
Add 1 -> 6
Write 6
```

Thread B:

```
Read count = 5
Add 1 -> 6
Write 6
```

Expected result after two increments: 7

Actual result: 6

One update was lost.

Race Condition Timeline

```
Initial count = 5

Thread A reads count    -> 5
Thread B reads count    -> 5
Thread A adds 1         -> 6
Thread B adds 1         -> 6
Thread A writes count   -> 6
```

```
Thread B writes count    -> 6

Final count = 6, but expected 7
```

Why Race Conditions Are Dangerous

- They may happen rarely.
- They depend on timing.
- They are hard to reproduce.
- They may disappear during debugging.
- They can corrupt data.
- They can cause security bugs.

Conditions for Race Condition

A race condition can happen when:

- There is shared data.
- Multiple threads or processes access it.
- At least one modifies it.
- Access is not properly synchronized.

10. Critical Section

A critical section is the part of a program where shared resources are accessed.

Examples of shared resources:

- Shared variable
- Shared file
- Shared database record
- Shared queue
- Shared memory region
- Printer

Example

```
// Entry section
lock(mutex);

// Critical section
count++;

// Exit section
unlock(mutex);

// Remainder section
```

Critical Section Problem

The critical section problem asks:

How can we design a protocol so that multiple processes or threads can safely share data?

A correct solution should satisfy three conditions:

1. Mutual exclusion
2. Progress
3. Bounded waiting

1. Mutual Exclusion

Only one process or thread can be inside the critical section at a time.

If Thread A is modifying shared data, Thread B must wait.

2. Progress

If no thread is inside the critical section, and some threads want to enter, the decision about who enters next cannot be postponed forever.

In simple words:

The system should not get stuck when the critical section is free.

3. Bounded Waiting

There must be a limit on how many times other threads can enter the critical section after a thread has requested entry.

In simple words:

No thread should starve forever.

11. Mutual Exclusion

Mutual exclusion means allowing only one thread or process to access a critical section at a time.

It prevents race conditions by making access to shared data controlled.

Common ways to achieve mutual exclusion:

- Mutex locks
- Semaphores
- Monitors
- Spinlocks
- Atomic operations

12. Mutex

A mutex is a locking mechanism used to provide mutual exclusion.

The word mutex means mutual exclusion.

Only one thread can hold a mutex at a time.

Basic Mutex Operations

```
lock(mutex)
unlock(mutex)
```

Mutex Flow

```
Thread wants critical section
    |
    v
Try to lock mutex
    |
    +-- If unlocked: acquire lock and enter
    |
    +-- If locked: wait
    |
    v
Execute critical section
    |
    v
Unlock mutex
```

Mutex Example

```
pthread_mutex_t lock;
int count = 0;

void *increment(void *arg) {
    pthread_mutex_lock(&lock);
    count++;
    pthread_mutex_unlock(&lock);
    return NULL;
}
```

Important Mutex Rule

The same thread that locks the mutex should unlock it.

This is one major difference between mutexes and semaphores.

Mutex Advantages

- Simple to understand
- Protects critical sections
- Prevents race conditions
- Good for exclusive access

Mutex Disadvantages

- Can cause deadlock
- Can reduce parallelism
- Incorrect unlock can cause bugs
- Holding lock too long hurts performance

When to Use Mutex

Use a mutex when:

- Only one thread should access a resource at a time.
- You need ownership semantics.
- You are protecting a critical section.

Example:

- Updating a shared counter
- Modifying a shared linked list
- Writing to a shared log buffer

13. Semaphore

A semaphore is a synchronization tool that uses a counter.

It controls access to one or more instances of a resource.

Semaphores were introduced by Edsger Dijkstra.

Basic Semaphore Operations

Traditional names:

```
wait(S)
signal(S)
```

Also called:

```
P(S)
V(S)
```

Modern names:

```
down(S)
up(S)
```

or:

```
acquire(S)
release(S)
```

wait Operation

The **wait** operation decreases the semaphore value.

If the value is not available, the thread blocks.

Conceptually:

```
wait(S):
    while S <= 0:
        wait
    S = S - 1
```

In a real OS, this must be atomic.

signal Operation

The **signal** operation increases the semaphore value.

If threads are waiting, one may be woken up.

Conceptually:

```
signal(S):
    S = S + 1
```

This must also be atomic.

Types of Semaphores

There are two common types:

1. Binary semaphore
2. Counting semaphore

Binary Semaphore

A binary semaphore can have only two values:

- 0
- 1

It is similar to a mutex, but not exactly the same.

Used for:

- Mutual exclusion
- Signaling between threads

Counting Semaphore

A counting semaphore can have values greater than 1.

It is used when there are multiple identical resources.

Example:

If a system has 3 printers, initialize semaphore to 3.

```
Semaphore printers = 3
```

Each process does:

```
wait(printers)
use printer
signal(printers)
```

At most 3 processes can use printers at the same time.

Semaphore Example

```
sem_t empty;

sem_wait(&empty);
// use resource
sem_post(&empty);
```

Mutex vs Semaphore

Feature	Mutex	Semaphore
Meaning	Mutual exclusion lock	Counter-based synchronization
Value	Locked or unlocked	Integer counter

Feature	Mutex	Semaphore
Ownership	Has ownership	Usually no ownership
Unlock/release	Same thread should unlock	Another thread may signal
Main use	Protect critical section	Control access or signal events
Resource count	One resource	One or many resources
Example	One thread updates list	Limit database connections to 10

Placement-Friendly Difference

A mutex is mainly used for mutual exclusion, where only one thread can enter a critical section and the thread that locks it should unlock it. A semaphore is a counter-based synchronization mechanism used either for mutual exclusion or for managing multiple instances of a resource. Unlike a mutex, a semaphore can be signaled by a different thread.

14. Condition Variables

A condition variable is a synchronization mechanism that allows threads to wait until a certain condition becomes true.

Condition variables are usually used with a mutex.

They do not protect shared data by themselves. The mutex protects the data. The condition variable helps threads sleep and wake up based on a condition.

Why Do We Need Condition Variables?

Suppose a consumer thread wants to remove an item from a queue.

If the queue is empty, the consumer cannot continue.

Bad approach:

```
while (queue_empty()) {  
    // keep checking  
}
```

This wastes CPU. It is called busy waiting.

Better approach:

```
Sleep until producer adds an item.
```

This is where condition variables are useful.

Basic Operations

Common operations:

```
wait(condition, mutex)
signal(condition)
broadcast(condition)
```

wait

wait does two things atomically:

1. Releases the mutex.
2. Puts the thread to sleep.

When the thread wakes up, it reacquires the mutex before continuing.

This atomic release-and-sleep behavior is very important. Without it, wakeups could be missed.

signal

signal wakes one waiting thread.

broadcast

broadcast wakes all waiting threads.

Correct Condition Variable Pattern

Always check the condition in a **while** loop, not an **if**.

```
pthread_mutex_lock(&mutex);

while (queue_empty()) {
    pthread_cond_wait(&not_empty, &mutex);
}

item = remove_from_queue();

pthread_mutex_unlock(&mutex);
```

Why Use while Instead of if?

Use **while** because:

- Spurious wakeups can happen.
- Another thread may consume the resource before this thread runs.
- The condition may no longer be true after waking.

Wrong:

```
if (queue_empty()) {  
    pthread_cond_wait(&not_empty, &mutex);  
}
```

Right:

```
while (queue_empty()) {  
    pthread_cond_wait(&not_empty, &mutex);  
}
```

Producer-Consumer Example

Producer:

```
pthread_mutex_lock(&mutex);  
  
add_item_to_queue(item);  
pthread_cond_signal(&not_empty);  
  
pthread_mutex_unlock(&mutex);
```

Consumer:

```
pthread_mutex_lock(&mutex);  
  
while (queue_empty()) {  
    pthread_cond_wait(&not_empty, &mutex);  
}  
  
item = remove_item_from_queue();  
  
pthread_mutex_unlock(&mutex);
```

Condition Variable vs Semaphore

Feature	Condition Variable	Semaphore
Stores count?	No	Yes
Used with mutex?	Usually yes	Not always
Main purpose	Wait for condition to become true	Count resources or signal

Feature	Condition Variable	Semaphore
If signal happens before wait	Signal may be lost	Count is remembered
Protects shared data?	No	Not directly

Important point:

A condition variable does not remember signals. If no thread is waiting when `signal` is called, the signal is lost. Therefore, the actual condition must be stored in shared state protected by a mutex.

15. Busy Waiting and Spinlocks

Busy waiting means repeatedly checking a condition without sleeping.

Example:

```
while (lock_is_taken) {  
    // spin  
}
```

This wastes CPU if the wait is long.

A spinlock is a lock where the waiting thread keeps spinning until the lock becomes available.

When Spinlocks Are Useful

Spinlocks can be useful when:

- Lock hold time is very short.
- Sleeping and waking overhead is greater than spinning.
- Used inside kernel code or low-level systems.

When Spinlocks Are Bad

Spinlocks are bad when:

- Lock may be held for a long time.
- The thread holding the lock may be descheduled.
- There is only one CPU core.

16. Atomic Operations

An atomic operation is an operation that completes as one indivisible unit.

No other thread can observe it halfway done.

Examples:

- Atomic increment
- Test-and-set
- Compare-and-swap
- Fetch-and-add

Atomic operations are used to build locks and lock-free data structures.

Compare-and-Swap

Compare-and-swap, or CAS, works like this:

```
CAS(address, expected, new_value):  
    if *address == expected:  
        *address = new_value  
        return true  
    else:  
        return false
```

CAS is atomic.

It is widely used in modern concurrent programming.

17. Deadlock

A deadlock is a situation where two or more processes or threads wait forever for resources held by each other.

Example

```
Thread A holds Lock 1 and waits for Lock 2.  
Thread B holds Lock 2 and waits for Lock 1.
```

Neither can proceed.

Four Necessary Conditions for Deadlock

Deadlock can occur only if all four conditions hold:

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

1. Mutual Exclusion

At least one resource must be non-shareable.

Example:

Only one thread can hold a mutex at a time.

2. Hold and Wait

A thread holds one resource while waiting for another.

3. No Preemption

Resources cannot be forcibly taken away.

They must be released voluntarily.

4. Circular Wait

There is a circular chain of threads, where each waits for a resource held by the next.

Deadlock Prevention

Prevent deadlock by breaking at least one of the four conditions.

Common techniques:

- Acquire locks in a fixed global order.
- Avoid holding one lock while waiting for another.
- Use timeout-based locking.
- Reduce lock scope.
- Use deadlock detection.

Lock Ordering Example

If every thread always acquires locks in this order:

```
Lock A -> Lock B -> Lock C
```

then circular wait is avoided.

18. Starvation

Starvation occurs when a thread waits indefinitely because other threads keep getting access to the resource.

Unlike deadlock, the system as a whole may still make progress.

Example:

A low-priority thread never runs because high-priority threads keep arriving.

Deadlock vs Starvation

Feature	Deadlock	Starvation
Meaning	Threads wait on each other forever	One thread waits indefinitely
System progress	Often no progress among deadlocked threads	Other threads may progress
Cause	Circular waiting	Unfair scheduling/resource allocation
Fix	Break deadlock conditions	Fair scheduling, aging

19. Livelock

A livelock occurs when threads are not blocked, but they keep responding to each other in a way that prevents progress.

They are active, but no useful work is done.

Example:

Two people in a hallway both move left, then both move right, repeatedly, but neither passes.

In threading:

Two threads repeatedly release and retry locks to avoid deadlock, but both keep retrying at the same time.

20. Classical Synchronization Problems

These problems are common in OS interviews.

20.1 Producer-Consumer Problem

Also called the bounded-buffer problem.

There is:

- A buffer of fixed size
- Producer threads that add items
- Consumer threads that remove items

Rules:

- Producer must wait if buffer is full.
- Consumer must wait if buffer is empty.
- Shared buffer must be protected.

Semaphore solution:

```
mutex = 1
empty = buffer_size
full = 0
```

Producer:

```
wait(empty)
wait(mutex)

add item to buffer

signal(mutex)
signal(full)
```

Consumer:

```
wait(full)
wait(mutex)

remove item from buffer

signal(mutex)
signal(empty)
```

Meaning:

- `mutex` protects the buffer.
- `empty` counts empty slots.
- `full` counts filled slots.

20.2 Readers-Writers Problem

Multiple readers can read shared data at the same time.

But writers need exclusive access.

Rules:

- Multiple readers may read together.
- Only one writer may write.
- Reader and writer cannot access shared data simultaneously.

Common issue:

- Reader preference can starve writers.
- Writer preference can starve readers.

20.3 Dining Philosophers Problem

Five philosophers sit around a table.

Each philosopher needs two forks to eat.

This problem demonstrates deadlock and resource allocation issues.

Deadlock can happen if every philosopher picks up the left fork and waits for the right fork.

Possible solutions:

- Allow at most four philosophers to sit at the table.
 - Pick up forks in a fixed order.
 - Use a waiter/arbitrator.
 - Make one philosopher pick up right fork first and left fork second.
-

21. Inter-Process Communication

Processes do not share memory by default.

They need inter-process communication, or IPC, to communicate.

Common IPC mechanisms:

- Pipes
- Message queues
- Shared memory
- Sockets
- Signals
- Files

Pipes

Pipes allow one process to send data to another.

Usually one-way.

Example:

```
ls | grep ".txt"
```

Output of `ls` goes into `grep`.

Message Queues

Processes exchange structured messages through a queue.

Useful when processes should not share memory directly.

Shared Memory

Multiple processes map the same memory region into their address spaces.

This is fast but requires synchronization.

Sockets

Sockets allow communication between processes, possibly on different machines.

Used in networking.

22. Thread Safety

A function or data structure is thread-safe if it behaves correctly when accessed by multiple threads at the same time.

Ways to make code thread-safe:

- Avoid shared mutable data.
- Use mutexes.
- Use atomic operations.
- Use immutable data.
- Use thread-local storage.
- Use message passing.

Reentrant Function

A reentrant function can be safely called again before a previous call finishes.

Usually, a reentrant function:

- Does not use shared mutable global state.
- Does not return pointers to static internal data.
- Does not depend on non-reentrant functions.

All reentrant functions are thread-safe in the relevant sense, but not all thread-safe functions are reentrant.

23. Common Synchronization Mistakes

Mistake 1: Forgetting to Unlock

```
lock(m);  
if (error) {  
    return;  
}  
unlock(m);
```

Problem:

If **error** is true, the mutex is never unlocked.

Better:

Use structured cleanup, **finally**, RAII, or defer-like patterns depending on language.

Mistake 2: Holding Lock Too Long

Bad:

```
lock
perform slow network call
unlock
```

Problem:

Other threads are blocked unnecessarily.

Better:

Do slow work outside the critical section if possible.

Mistake 3: Nested Locks Without Ordering

```
Thread A: lock X, then lock Y
Thread B: lock Y, then lock X
```

This can deadlock.

Mistake 4: Using if Instead of while With Condition Variables

Wrong:

```
if (!condition) {
    wait(cond, mutex);
}
```

Right:

```
while (!condition) {
    wait(cond, mutex);
}
```

Mistake 5: Thinking volatile Solves Race Conditions

In languages like C and C++, `volatile` does not generally make operations atomic and does not replace proper synchronization.

Use mutexes, atomics, or language-specific synchronization primitives.

24. Interview Questions and Answers

Q1. What is the difference between a process and a thread?

A process is an independent program in execution with its own address space and resources. A thread is a lightweight execution unit within a process. Threads share the process address space and resources, but each thread has its own stack, registers, and program counter. Processes provide stronger isolation, while threads are cheaper and communicate faster.

Q2. Why are threads called lightweight processes?

Threads are called lightweight processes because they have their own execution state but share most resources with other threads in the same process. Creating and switching threads is usually cheaper than creating and switching processes.

Q3. What is a race condition?

A race condition occurs when multiple threads or processes access shared data concurrently and the final result depends on the timing or order of execution. It usually happens when at least one thread modifies shared data without proper synchronization.

Q4. What is a critical section?

A critical section is a part of code where shared data or shared resources are accessed. Since incorrect concurrent access can cause race conditions, only one thread or process should execute a critical section at a time.

Q5. What are the requirements of a critical section solution?

A correct critical section solution should satisfy:

- Mutual exclusion
- Progress
- Bounded waiting

Q6. What is a mutex?

A mutex is a lock used to provide mutual exclusion. Only one thread can hold the mutex at a time. It is commonly used to protect critical sections.

Q7. What is a semaphore?

A semaphore is a synchronization primitive based on an integer counter. It supports wait and signal operations. It can be used to control access to multiple instances of a resource or to signal between threads.

Q8. Difference between mutex and semaphore?

A mutex is used mainly for mutual exclusion and has ownership, meaning the thread that locks it should unlock it. A semaphore is counter-based and may allow multiple threads to access multiple resource instances. A semaphore can also be signaled by a different thread.

Q9. What is a condition variable?

A condition variable allows a thread to sleep until a particular condition becomes true. It is usually used with a mutex. The mutex protects the shared state, while the condition variable handles waiting and waking.

Q10. Why should condition variable waits be inside a while loop?

Because spurious wakeups can happen, and because another thread may change the condition before the awakened thread gets the mutex. The thread must recheck the condition after waking.

Q11. What is deadlock?

Deadlock is a situation where two or more threads or processes wait forever for resources held by each other.

Q12. What are the four conditions for deadlock?

The four necessary conditions are:

- Mutual exclusion
- Hold and wait
- No preemption
- Circular wait

Q13. What is starvation?

Starvation occurs when a thread waits indefinitely because other threads keep getting scheduled or keep acquiring the needed resource.

Q14. What is the difference between concurrency and parallelism?

Concurrency means multiple tasks make progress during the same time period. Parallelism means multiple tasks execute at the exact same time, usually on multiple CPU cores.

Q15. What happens during a context switch?

The OS saves the state of the currently running process or thread and loads the saved state of another. This includes registers, program counter, stack pointer, and scheduling state.

25. Quick Revision Tables

Process, Thread, Coroutine

Concept	Description	Managed by	Memory
Process	Program in execution	OS	Separate address space
Thread	Execution unit inside process	OS or thread library	Shared process memory
Coroutine	Cooperative execution unit	Program/runtime	Usually same thread memory

Synchronization Tools

Tool	Best For	Key Idea
Mutex	Protecting critical section	One owner at a time
Binary semaphore	Mutual exclusion or signaling	Counter is 0 or 1
Counting semaphore	Multiple identical resources	Counter tracks availability
Condition variable	Waiting for a condition	Sleep until state changes
Spinlock	Very short critical sections	Busy wait
Atomic variable	Small lock-free updates	Indivisible operation

Common Problems and Fixes

Problem	Meaning	Common Fix
Race condition	Timing affects correctness	Mutex, atomic, semaphore
Deadlock	Threads wait forever	Lock ordering, timeout
Starvation	Thread waits indefinitely	Fair scheduling, aging
Livelock	Active but no progress	Random backoff, better protocol
Busy waiting	Wastes CPU while waiting	Blocking wait, condition variable

26. One-Minute Summary

A process is an independent program in execution with its own address space. A thread is a lightweight execution unit inside a process. Threads share memory, so communication is fast, but shared data creates synchronization problems. A race condition happens when the result depends on unpredictable timing. A critical section is code that accesses shared resources. Mutexes provide exclusive access. Semaphores use counters to control resources or signal events. Condition variables let threads sleep until a condition becomes true and are almost always used with a mutex. Correct concurrent programs must avoid race conditions, deadlocks, starvation, and incorrect signaling.

27. How to Answer in Interviews

When asked about any concurrency concept, use this structure:

1. Define it clearly.
2. Explain why it is needed.
3. Give one simple example.
4. Mention one common issue or tradeoff.

Example for mutex:

"A mutex is a mutual exclusion lock used to protect a critical section. It is needed when multiple threads share mutable data. For example, if two threads increment the same counter, a mutex ensures only one thread updates it at a time. The tradeoff is that incorrect use can cause deadlock or reduce parallelism."

28. Must-Remember Points

- Process has separate memory; thread shares memory.
- Threads are faster but less isolated.
- Shared mutable data needs synchronization.
- `count++` is not automatically atomic.
- Critical section means shared resource access code.
- Mutex gives exclusive access.
- Semaphore is a counter.
- Condition variable is for waiting until a condition becomes true.
- Always use `while`, not `if`, with condition variable waits.
- Deadlock requires mutual exclusion, hold and wait, no preemption, and circular wait.
- Avoid deadlock with consistent lock ordering.
- Concurrency is about structure; parallelism is about simultaneous execution.